IEAP-Series03-Assignment

Sarah Durand, Gaia Gassiole, Vidusha Thebuwana

2026-10-11

GitHub repository: https://github.com/sdurand875-spec/IEAP-Series03-RStudio

IEAP-Rstudio-Series03

Data wrangling and ANOVA

1 ANOVA: Analysis of variance

1.1 Think before coding

The scientific question is:

Is the effect of difficulty on Movement Time the same across all groups?

More specifically, we aim to investigate how the effect of the Index of Difficulty (ID) on Movement Time differs between the three groups: stroke patients, age-matched controls, and young controls.

We hypothesize that the effect of difficulty on Movement Time is stronger in the post-stroke patient group than in the two control groups.

How do we visualize this?

We use graphs created with `ggplot2` in R to plot Movement Time against the Index of Difficulty for each group. The regression lines allow us to compare the relationship between difficulty and Movement Time across groups.

Is there a significant difference?

We use an ANOVA to test whether the effect of difficulty on Movement Time differs between groups. The significance threshold is set at $= 0.01$.

What conclusion can we draw?

The ANOVA revealed a significant GROUP × ID interaction ($F(2, 48) = 11.58$, $p < 0.01$), indicating that the effect of difficulty on Movement Time differs significantly between the three groups. As shown in the graph, Movement Time increases with difficulty in all three groups, but this increase is much stronger in the post stroke patients' group than in the two control groups.

1.2 What library to answer the above questions?

Load library

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.2.1     v readr     2.2.0
v forcats   1.0.1     v stringr   1.6.0
v ggplot2   4.0.3     v tibble    3.3.1
v lubridate 1.9.5     v tidyr     1.3.2
v purrr     1.2.2
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to beco
```

```
library(ez)
```

```
Registered S3 method overwritten by 'lme4':
  method           from
  na.action.merMod car
```

1.3 Data reorganization

Load data

```r
data <- read.table(
  text = readLines("data/Results.txt"),
  header = TRUE,
  sep = "",
  stringsAsFactors = TRUE
)

head(data)
```

```
  GROUP  SUBJ TRIAL ORI     DIR  ID REP NbVelPeaks MovementTime
1    ap apS01   T01   W outward ID1  R1          2         1.23
2    ap apS01   T01   W  inward ID1  R1          2         1.26
3    ap apS01   T01  NW outward ID1  R1          3         1.14
4    ap apS01   T01  NW  inward ID1  R1          2         0.90
5    ap apS01   T01   N outward ID1  R1          3         1.11
6    ap apS01   T01   N  inward ID1  R1          3         0.81
```

We need to reorganize the data.

To do so, we count the number of data points for each combination of factors to check if there is missing data.

The following function looks for every unique combination of GROUP and ID in the dataset and tallies up the number of rows for each pair. It automatically generates a new column named n that contains these totals. we do that to check if the number of rows is the same for every combination.

```r
data_counts <- data %>%
  count(GROUP, ID)

print(data_counts)
```

```
  GROUP  ID    n
1    ap ID1  740
2    ap ID2  740
3    ap ID3  740
4    cp ID1 1140
5    cp ID2 1140
6    cp ID3 1140
7    pp ID1  760
8    pp ID2  760
9    pp ID3  400
```

For item 9 “pp ID3 400”, the number of rows does not match the rest of the group ‘PP’, so we need to delete ID3. The fact that some data are missing is explained by some stroke patients not being able to perform the test at ID3.

```
data %>%
  filter(ID != "ID3") %>%
  count(GROUP, ID)
```

```
  GROUP  ID    n
1    ap ID1  740
2    ap ID2  740
3    cp ID1 1140
4    cp ID2 1140
5    pp ID1  760
6    pp ID2  760
```

Now let’s look at other columns such as REP.

```
data %>%
  filter(ID != "ID3") %>%
  count(REP)
```

```
  REP    n
1  R1 2040
2  R2 2040
3  R3 1200
```

The number of repetitions for R3 is half the number of R1 and R2. Therefore, we delete R3 data from our data frame.

```
data %>%
  filter(ID != "ID3") %>%
  filter(REP != "R3") %>%
  count(GROUP, ID, REP)
```

```
   GROUP  ID REP   n
1     ap ID1  R1 260
2     ap ID1  R2 260
3     ap ID2  R1 260
4     ap ID2  R2 260
```

```
5     cp ID1  R1 380
6     cp ID1  R2 380
7     cp ID2  R1 380
8     cp ID2  R2 380
9     pp ID1  R1 380
10    pp ID1  R2 380
11    pp ID2  R1 380
12    pp ID2  R2 380
```

We created a tibble 3×2 to look for the number of participants in each group.

```
data %>%
  filter(ID != "ID3") %>%
  filter(REP != "R3") %>%
  distinct(GROUP, SUBJ) %>%
  count(GROUP, name='count')
```

```
  GROUP count
1    ap    13
2    cp    19
3    pp    19
```

Now we create a new variable to store the cleaned data.

```
data_clean <- data %>%
  filter(ID != "ID3") %>%
  filter(REP != "R3")
head(data_clean)
```

```
  GROUP  SUBJ TRIAL ORI     DIR  ID REP NbVelPeaks MovementTime
1    ap apS01   T01   W outward ID1  R1          2         1.23
2    ap apS01   T01   W  inward ID1  R1          2         1.26
3    ap apS01   T01  NW outward ID1  R1          3         1.14
4    ap apS01   T01  NW  inward ID1  R1          2         0.90
5    ap apS01   T01   N outward ID1  R1          3         1.11
6    ap apS01   T01   N  inward ID1  R1          3         0.81
```

Run the ANOVA

```r
anova_results <- ezANOVA(
  data = data_clean,
  dv = .(MovementTime),# dependent variable
  wid = .(SUBJ),# to recognize each participant
  within = .(ID),# within-subject factor
  between = .(GROUP), # between-subject factor
  detailed = FALSE
)
```

```
Warning: You have removed one or more levels from variable "ID". Refactoring
for ANOVA.
```

```
Warning: Data is unbalanced (unequal N per group). Make sure you specified a
well-considered value for the type argument to ezANOVA().
```

```
Warning: Collapsing data to cell means. *IF* the requested effects are a subset
of the full design, you must use the "within_full" argument, else results may
be inaccurate.
```

```r
anova_results$ANOVA
```

```
    Effect DFn DFd        F            p p<.05        ges
2    GROUP   2  48 37.24408 1.720007e-10     * 0.57462110
3       ID   1  48 30.77618 1.221240e-06     * 0.07667596
4 GROUP:ID   2  48 11.58124 7.866206e-05     * 0.05882293
```

ANOVA p-value:

We format the p-value to avoid scientific notation and see the full number. That way, it's easier to interpret the results.

```r
format(anova_results$ANOVA$p, scientific = FALSE, digits = 10)
```

```
[1] "0.000000000172000706" "0.000001221240178853" "0.000078662063814274"
```

Interpretation:

The ANOVA revealed significant main effects of GROUP and ID, as well as a significant GROUP × ID interaction, with all p-values below 0.01.

Detailed ANOVA results:

Effect of the GROUP: Movement Time is different between the three groups ($p < 0.01$).

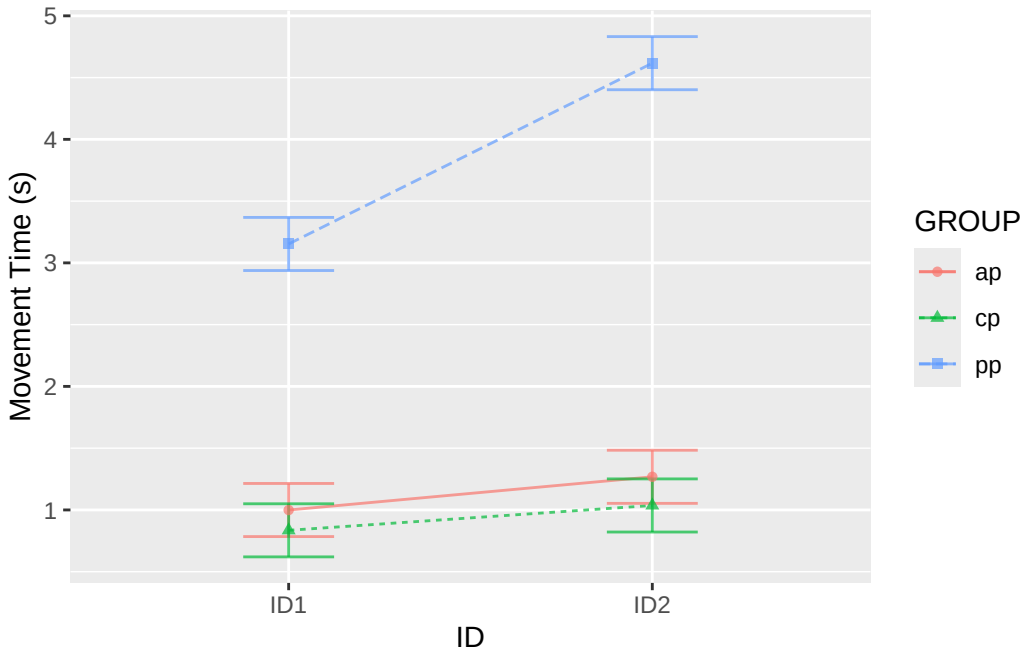
Effect of ID: Movement Time is different between the two IDs ($p < 0.01$).

Interaction effect GROUPxID: The effect of ID on Movement Time depends on the GROUP ($p < 0.01$). That is to say, Movement Time differs between the groups depending on the ID. For now, we do not know the direction of the effect, but we will see it in the graph below.

Plot Movement Time as a Function of Index of Difficulty

```
interaction_plot_reference <- ggplot(data_clean) +
  aes(x = ID, y = MovementTime, color = GROUP) +
  # lines: one line type per group
  stat_summary(fun = mean, geom = "line",
               aes(group = GROUP, linetype = GROUP), alpha = 0.7) +
  # error bars: mean ± 0.215 s, same size for all groups
  stat_summary(fun.data = function(x) data.frame(y    = mean(x),
                                                 ymin = mean(x) - 0.215,
                                                 ymax = mean(x) + 0.215),
               geom = "errorbar", width = 0.25, alpha = 0.7) +
  # points: one shape per group
  stat_summary(fun = mean, geom = "point",
               aes(shape = GROUP), size = 1.5, alpha = 0.7) +
  labs(x = "ID", y = "Movement Time (s)")

show(interaction_plot_reference)
```



Interpretation:

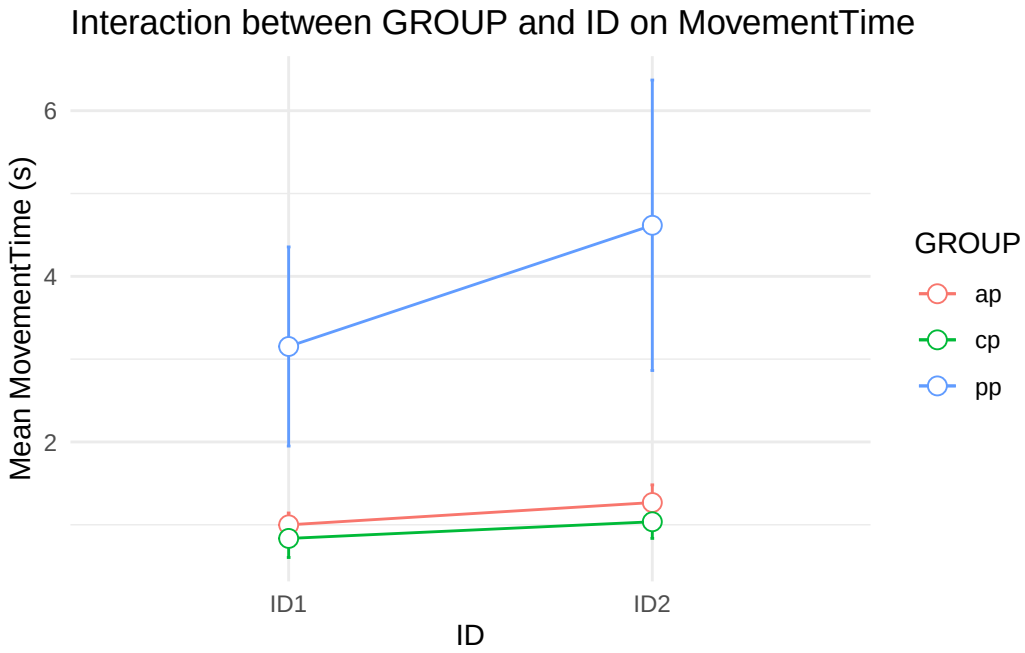
From the ANOVA analysis, we already know that there is a significant interaction effect between GROUP and ID. From the graph, we can see that Movement Time increases as the Index of Difficulty increases in all three groups. However, this increase is much stronger in the stroke patients group compared to the two other groups.

Continuation of the section 1.3

```
library(ggplot2)
```

```
# Interaction between GROUP and ID, with error bars of +/- 0.5 SD around the mean
interaction_plot_Group_ID <- ggplot(data_clean) +
  aes(x = ID, y = MovementTime, color = GROUP) +
  stat_summary(fun = mean, geom = "line", aes(group = GROUP)) +
  stat_summary(fun.data = mean_sdl, fun.args = list(mult = .5),
               geom = "errorbar", width = 0.01) +
  stat_summary(fun = mean, geom = "point", shape = 21, fill = "white", size = 3) +
  labs(title = "Interaction between GROUP and ID on MovementTime",
       x = "ID",
       y = "Mean MovementTime (s)") +
  theme_minimal()
```

```
print(interaction_plot_Group_ID)
```



1.4 Run the linear regression by GROUP

First let's check the data types of each column. Linear regression requires a number not strings.

```
str(data_clean)
```

```
'data.frame':   4080 obs. of  9 variables:
 $ GROUP       : Factor w/ 3 levels "ap","cp","pp": 1 1 1 1 1 1 1 1 1 1 ...
 $ SUBJ        : Factor w/ 51 levels "apS01","apS02",..: 1 1 1 1 1 1 1 1 1 1 ...
 $ TRIAL       : Factor w/ 2 levels "T01","T02": 1 1 1 1 1 1 1 1 1 1 ...
 $ ORI         : Factor w/ 5 levels "E","N","NE","NW",..: 5 5 4 4 2 2 3 3 1 1 ...
 $ DIR         : Factor w/ 2 levels "inward","outward": 2 1 2 1 2 1 2 1 2 1 ...
 $ ID          : Factor w/ 3 levels "ID1","ID2","ID3": 1 1 1 1 1 1 1 1 1 1 ...
 $ REP         : Factor w/ 3 levels "R1","R2","R3": 1 1 1 1 1 1 1 1 1 1 ...
 $ NbVelPeaks  : num  2 2 3 2 3 3 1 2 4 1 ...
 $ MovementTime: num  1.23 1.26 1.14 0.9 1.11 0.81 1.08 0.81 1.32 1.02 ...
```

Since ID is a categorical variable and it is better to run the actual numeric values before fitting to the linear model. Since ID1 was 3.8 and ID2 was 5.43 from the scientific paper, the data can be manipulated as follows:

```
data_reg <- data_clean %>%
  # Create a new column called 'ID_num', ID1 is 3.8, ID2 is 5.43
  mutate(ID_num = case_when(
    ID == "ID1" ~ 3.8,
    ID == "ID2" ~ 5.43
  ))
```

Now that data type is stored properly. Let's run the linear model.

```
models <- data_reg %>%
  # Split the data into separate group based on the 'GROUP' column
  group_by(GROUP) %>%
  # Loop through each pile (.x) and calculate a linear model (lm) predicting MovementTime fr
  group_map(~ lm(MovementTime ~ ID_num, data = .x))

# Apply the actual group names (e.g., ap, cp, pp) as labels to the model list
names(models) <- levels(data_reg$GROUP)

# Loop that goes through each group name one by one (where 'g' represents the current name)
for (g in names(models)) {
  # Extract the math coefficients (intercept and slope) from the model belonging to group 'g
  co <- coef(models[[g]])
  # Print the final equation as text, with 3 decimal places
  cat("Group", g, ": MT =", round(co[1], 3), "+", round(co[2], 3), "* ID\n")
}
```

```
Group ap : MT = 0.372 + 0.165 * ID
Group cp : MT = 0.366 + 0.123 * ID
Group pp : MT = -0.259 + 0.898 * ID
```

Interpretation of the data:

Group	Intercept (b)	Slope (a)	Meaning of the slope
ap	0.372 s	0.165 s/bit	each extra bit of difficulty adds about 0.17 s

Group	Intercept (b)	Slope (a)	Meaning of the slope
cp	0.366 s	0.123 s/bit	each extra bit adds about 0.12 s
pp	-0.259 s	0.898 s/bit	each extra bit adds about 0.90 s

The pp group adds around 0.9s per unit increase in ID. This figure is much larger compared to the figures of ap(0.17s) and cp(0.12s).

1.5 Save a graph in a PDF file for publication

Construction of the graph

To reproduce a graph similar to Figure 3 of the article, we progressively construct the figure from the cleaned data.

First, we calculate the **mean Movement Time** and the **standard deviation** for each combination of group (`ap`, `cp`, and `pp`) and difficulty level (`3.8` and `5.43` bits). This gives us six mean values, corresponding to the six points represented in the graph.

We use `ID_num` to represent the Index of Difficulty numerically on the x-axis.

The following table summarizes the variables used to construct the graph:

Variable	Role in the graph
`GROUP`	Defines the three groups: `ap`, `cp`, and `pp`
`ID_num`	Index of Difficulty on the x-axis: 3.8 and 5.43 bits
`MovementTime`	Movement Time measured for each trial
`mean_MovementTime`	Mean Movement Time represented by each point
`sd_MovementTime`	Standard deviation used to construct the error bars

We then use `ggplot2` to represent the six mean values. The Index of Difficulty is placed on the x-axis, the mean Movement Time on the y-axis, and a different colour is used for each group.

A linear regression line is then fitted separately for each group. The `geom_smooth()` function uses the `lm` method, which corresponds to R's linear model function. It automatically estimates the slope and intercept of the best-fitting linear regression line from the data.

Each regression line follows the equation:

$$\mathbf{y = a \times x + b}$$

Letter	Mathematical name	In our graph
y	Measured variable	MT, the Movement Time
x	Explanatory variable	ID, the Index of Difficulty
a	Slope	Estimated by `lm()`
b	Intercept	Estimated by `lm()`

Therefore, for each group, the regression equation can be written as:

$$\mathbf{MT = b + a \times ID}$$

The regression coefficients obtained in Section 1.4 are:

Group	Intercept (b)	Slope (a)
ap	0.372	0.165
cp	0.366	0.123
pp	-0.259	0.898

These values give the following regression equations:

Group	Regression equation
ap	MT = 0.372 + 0.165 × ID
cp	MT = 0.366 + 0.123 × ID
pp	MT = -0.259 + 0.898 × ID

We add these equations to the graph using `annotate()`. They are positioned in the upper-left area of the graph, where there is enough free space.

Finally, the graph is modified to resemble Figure 3 of the article more closely. The axes and regression lines are extended, the two mean values of each group are connected with a solid line, error bars are added to represent variability, and the mean values are displayed as white circles with coloured outlines. The axis labels and overall theme are also adjusted.

```
# ============================================================
# Step 1: Calculate the mean and standard deviation
# ============================================================

# Compute the mean and standard deviation of Movement Time
# for each group and difficulty level.
group_means <- data_reg %>%
  group_by(GROUP, ID_num) %>%
```

```
  summarise(
    mean_MovementTime = mean(MovementTime),
    sd_MovementTime = sd(MovementTime),
    .groups = "drop"
  ) %>%
  # Reorder the groups (pp first) so that the colours match Figure 3 of the article
  mutate(GROUP = factor(GROUP, levels = c("pp", "ap", "cp")))

# Display the resulting table
# 6 rows = 3 groups x 2 difficulty levels
group_means
```

```
# A tibble: 6 x 4
  GROUP ID_num mean_MovementTime sd_MovementTime
  <fct>  <dbl>             <dbl>           <dbl>
1 ap      3.8              0.999           0.285
2 ap      5.43             1.27            0.427
3 cp      3.8              0.835           0.460
4 cp      5.43             1.04            0.401
5 pp      3.8              3.15            2.40
6 pp      5.43             4.62            3.51
```

```
# ============================================================
# Step 2: Create the basic graph
# ============================================================

fitts_plot <- ggplot(
  data = group_means,
  aes(
    x = ID_num,
    y = mean_MovementTime,
    color = GROUP
  )
) +
  geom_point()


# ============================================================
# Step 3: Add the regression equations
# ============================================================
```

```r
# The regression coefficients were obtained in Section 1.4.
# The equations are added manually using annotate().

fitts_plot <- fitts_plot +
  annotate(
    "text",
    x = 0,
    y = 6.5,
    label = "ap: MT = 0.372 + 0.165 * ID",
    hjust = 0,
    color = "#00BA38"
  ) +
  annotate(
    "text",
    x = 0,
    y = 6,
    label = "cp: MT = 0.366 + 0.123 * ID",
    hjust = 0,
    color = "#619CFF"
  ) +
  annotate(
    "text",
    x = 0,
    y = 5.5,
    label = "pp: MT = -0.259 + 0.898 * ID",
    hjust = 0,
    color = "#F8766D"
  )


# ============================================================
# Step 4: Improve the graph to resemble Figure 3
# ============================================================

# Extend the axes using invisible points.
fitts_plot <- fitts_plot +
  geom_blank(
    data = data.frame(
      x = c(-0.5, 6.5),
      y = c(-1, 7)
    ),
    aes(x = x, y = y),
```

```
    inherit.aes = FALSE
  ) +

  # Extend the regression lines to the edges of the graph
  # and display them as dashed lines.
 geom_smooth(
  data = data_reg,
  mapping = aes(
    x = ID_num,
    y = MovementTime,
    color = GROUP,
    group = GROUP
  ),
  method = "lm",
  se = FALSE,
  fullrange = TRUE,
  linetype = "dashed",
  linewidth = 0.3,
  inherit.aes = FALSE
) +

  # Connect the two mean values of each group.
  geom_line() +

  # Add error bars using half of the standard deviation.
  geom_errorbar(
    aes(
      ymin = mean_MovementTime - 0.5 * sd_MovementTime,
      ymax = mean_MovementTime + 0.5 * sd_MovementTime
    ),
    width = 0
  ) +

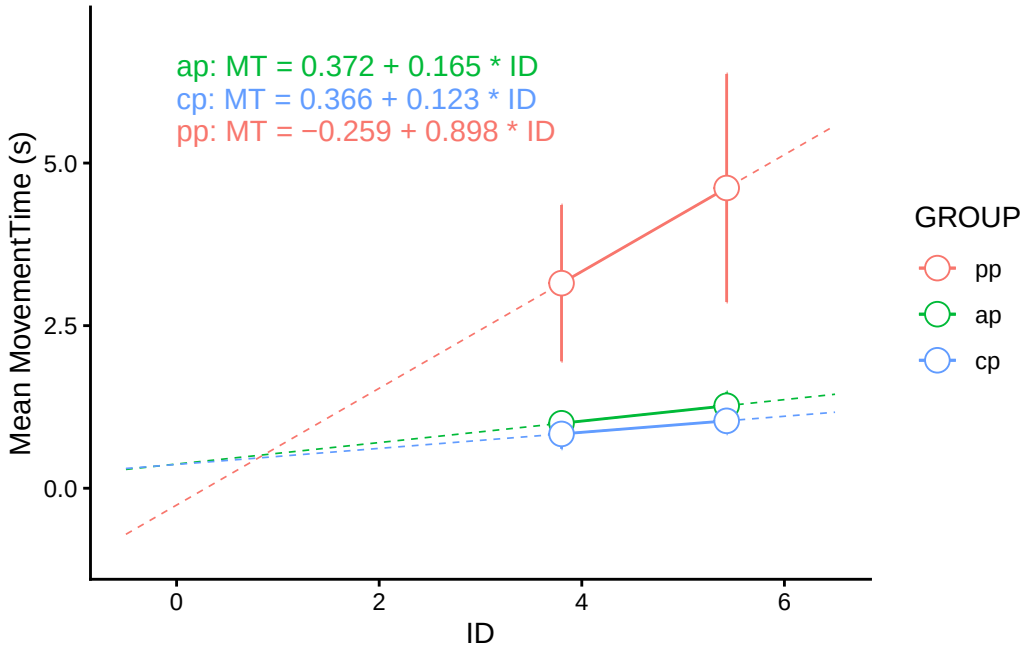
  # Display the mean values as white circles
  # with coloured outlines.
  geom_point(
    shape = 21,
    fill = "white",
    size = 4
  )
```

```r
# ============================================================
# Step 5: Final formatting
# ============================================================

fitts_plot <- fitts_plot +
  labs(
    x = "ID",
    y = "Mean MovementTime (s)"
  ) +
  theme_classic()

# Display the final graph
fitts_plot
```

```
`geom_smooth()` using formula = 'y ~ x'
```



Note on the code structure

The regression lines are extended to the edges of the graph using `geom_smooth()` with `fullrange = TRUE`. They are displayed as dashed lines to distinguish them from the solid lines connecting the observed group means.

1.6 Export the graph as a PDF file for publication

```
ggsave(
  filename =here::here("Figure3_MT_vs_ID.pdf"),
  plot = fitts_plot,
  width =8,
  height =6,
  units = "in"
)
```

```
`geom_smooth()` using formula = 'y ~ x'
```

1.7 Discuss the differences with the article

Our results may differ slightly from those reported in the article because we used a simpler analysis method.

First, we excluded observations corresponding to ID3 and repetition R3 during data cleaning. Second, we fitted separate linear regressions for each group using the trials pooled across participants, rather than fitting a separate regression for each participant. Finally, we represented variability using error bars corresponding to ± 0.5 standard deviations.

Despite these methodological differences, our results are consistent with the main conclusion of the article. Movement Time increases with the Index of Difficulty in all three groups, but the increase is much stronger in the post-stroke patient group (pp; slope 0.90 s/bit) than in the two control groups (ap and cp; slopes 0.17 and 0.12 s/bit, respectively). The significant GROUP $\times$ ID interaction in the ANOVA supports the conclusion that the effect of difficulty differs between groups.

However, our analysis has limitations. In particular, using only two difficulty levels does not allow us to assess the linearity of the relationship over a wider range of difficulty levels. Moreover, pooling trials across participants does not account for individual differences in the regression slopes. Therefore, although our results support a similar overall interpretation, they should not be considered an exact replication of the analysis presented in the article.

Git workflow strategy

Repository organisation

Sarah Durand created the public GitHub repository `IEAP-Series03-RStudio` and set up its structure. The project is organised around a **master Quarto document** (`IEAP-Series03-Rstudio.qmd`) and **sub-documents** stored in the `Section/` folder:

File	Content	Author
`IEAP-Series03-Rstudio.qmd`	Master document: header, data loading and cleaning, includes	Sarah Durand
`Section/section_1_gaia.qmd`	1.3 ANOVA and interaction plot	Gaia Gassiole
`Section/section_2_vidusha.qmd`	1.4 Linear regression by group	Vidusha Thebuwana
`Section/section_3_sarah.qmd`	1.5 Graph for publication and 1.6 PDF export	Sarah Durand

The sub-documents are inserted into the master document with `{{< include >}}` when rendering. This structure allowed each team member to work on her own file without modifying the files of the others, which limited conflicts.

Use of branches

Each team member worked on a **dedicated branch**:

- `Section_Gaia` for question 1.3,
- `Section_Vidusha` for question 1.4,
- `Section_Sarah` for questions 1.5 and 1.6,
- `Workflow` for the documentation of our workflow.

The `main` branch was kept as the stable version of the report. Working on separate branches allowed us to progress on different questions in parallel, and to test our code without breaking the work of the others.

Merging with Pull Requests

When a section was ready, it was merged into `main` through a **Pull Request** on GitHub (Pull Requests #1, #2 and #3). Before each merge, the branch was updated with the latest version of `main` (`git pull` and `git merge main`), so that each section could build on the previous ones (for example, section 1.5 uses `data_reg` created in section 1.4).

Commits

We made small and frequent commits, with meaningful messages describing each step (for example *"Compute mean Movement Time by group and ID"* or *"Section 1.6: export the graph as an 8 x 6 inch PDF"*), with at least one commit per question.

Integration and final corrections

Once the three sections were merged, Sarah Durand was in charge of the **integration of the whole report**. When rendering the master document, several problems appeared, and she corrected them:

- the data path in section 1.3 (`"../data/Results.txt"`) only worked from the `Section/` folder: it was replaced by `here::here("data", "Results.txt")`, which works both interactively and when rendering;
- duplicated chunk labels between the master document and section 1.3 stopped the rendering: the duplicated code was removed, as it was already in the master document;
- the full `data_clean` table was printed in the report: it was replaced by `head(data_clean)`;
- the content of the report was moved into `IEAP-Series03-Rstudio.qmd`, the master document required by the assignment;
- the date, the link to the GitHub repository and the `LICENSE` file were added, and the final PDF was rendered.

What worked well

- One file and one branch per person avoided most merge conflicts.
- Pull Requests gave the team the opportunity to review each section before merging it into `main`.
- The master/sub-document structure made the final report easy to assemble.

#Challenges encountered and lessons learned

1. Using Git as a team

Challenges

- **Keeping each branch up to date.** Several times, `git pull` did not work because local files had been modified (for example `.DS_Store`, a hidden file created automatically by macOS, or a section file modified by mistake). As a result, the local `main` branch was not up to date, and the work of the other team members did not appear.

- **Merging in the right order.** After a Pull Request was merged, new commits pushed on the same branch were not automatically in `main`: a new Pull Request was needed.
- **Typing errors in the terminal.** Two commands pasted on the same line (for example `git pullgit checkout -b ...`) were not executed, which created confusion.

Lessons learned

- Always run `git pull` on `main` before creating a new branch, and `git merge main` before opening a Pull Request.
- Check `git status` before each commit, and only add the files that were actually modified (never `.DS_Store`).
- Type Git commands one at a time, and read the message returned by Git before going further.

2. Rendering the master document

Challenges

When the three sections were assembled, the master document did not render, even though each section worked on its own. Sarah Durand identified and fixed the causes one by one:

- **Relative paths.** The path `"../data/Results.txt"` worked when running section 1.3 from the `Section/` folder, but not when rendering the master document from the main folder. It was replaced by `here::here("data", "Results.txt")`.
- **Duplicated code and chunk labels.** Section 1.3 repeated the data loading and cleaning code of the master document, with the same chunk labels, which stopped the rendering and displayed the same tables twice. The duplicated code was removed.
- **Unsaved files.** A correction did not appear in the report because the file had not been saved before rendering.
- **Missing packages.** Some packages (`ez`, `Hmisc`, `here`) had to be installed on each computer.

Lessons learned

- Each sub-document should only contain its own part, and build on the objects created before it (`data_clean`, `data_reg`).
- Use `here::here()` for file paths, so that the code works from any folder.
- Give unique names to code chunks, or no names at all.
- Save all files and render the master document regularly, not only at the end.

3. Building the graph with ggplot2

Challenges

- **Understanding the target figure.** Reproducing Figure 3 required first understanding what each element represents: the means, the regression lines, the error bars (mean ± 0.5 SD) and the equations.
- **Layers adding up.** Because the graph was built step by step (`fitts_plot <- fitts_plot + ...`), running the same chunk several times added the same layers again (for example the equations written on top of each other).
- **Colours and order of the groups.** By default, ggplot orders the groups alphabetically. To obtain the same colours as in the article (pp in red), the order of the factor levels had to be changed with `factor()`.
- **Axis limits.** Extending the regression lines beyond the data required `geom_blank()` combined with `fullrange = TRUE`.

Lessons learned

- Building a graph layer by layer makes each step easier to understand and to explain.
- When a graph is built step by step, the chunks must be run in order, each one only once.
- Colours in ggplot depend on the order of the factor levels.

4. Organisation and integration of the report

Challenges

- **Wrong master document.** The content of the report was first written in a file named `IEAP Series 03 Assignment.qmd`, whereas the assignment required the master document to be `IEAP-Series03-Rstudio.qmd`. In addition, spaces in the file name caused problems when rendering (Quarto created several output folders, and the HTML file did not always show the latest version).
- **Assembling work written separately.** Each section had been written and tested on its own, in its own folder. Once included in the master document, they did not fit together: some code was repeated, some paths no longer worked, and some objects were created twice.
- **Reviewing and correcting the work of others.** To obtain a report that renders, Sarah Durand had to read the sections written by the other team members, understand how their code worked, and identify the errors. The difficulty was to **correct these errors without creating new ones**, and without losing any of their work. For example:
  - in section 1.3, the data loading and cleaning code was a copy of the master document. Before removing it, we checked that the master document already loaded the libraries (`tidyverse`, `ez`) and created `data_clean`, so that the ANOVA of section 1.3 would still work;

  - the data path was corrected so that it works both when running the section alone and when rendering the whole report;
  - the report was rendered again after each correction, to check that nothing else was broken.

- **Keeping a clean and readable report.** Duplicated code and duplicated tables made the report long and confusing (the same cleaning steps and the full `data_clean` table appeared twice). These duplicates were removed, so that each step of the analysis appears only once, in a logical order.

Lessons learned

- Read the grading criteria carefully at the beginning of the project, and set up the right file names from the start (without spaces).
- Agree as a team, from the start, on **what goes in the master document and what goes in each section**, so that nobody repeats the work of the others.
- Before modifying someone else's code, **understand what it does and what the rest of the report depends on**, and make one correction at a time.
- **Render the full report after each correction**, to check immediately that the change did not create a new error.
- Inform the team when modifying their files (for example in the Pull Request), so that everyone knows what was changed and why.
- Integration takes time: it should be planned before the deadline, not only at the end.

Data sources and references

Data source

- `Results.txt`: movement time data from a Fitts' pointing task performed by three groups of participants (ap, cp, pp), provided by Denis Mottet for the IEAP RStudio course. Available at: https://moodle.umontpellier.fr/course/view.php?id=41022

Scientific references

- Mottet, D., van Dokkum, L. E. H., Froger, J., Gouaïch, A., & Laffont, I. (2017). Trajectory formation principles are the same after mild or moderate stroke. *PLoS ONE*, 12(3), e0173674. https://doi.org/10.1371/journal.pone.0173674
- Fitts, P. M. (1954). The information capacity of the human motor system in controlling the amplitude of movement. *Journal of Experimental Psychology*, 47(6), 381–391. https://doi.org/10.1037/h0055392

Software and R packages

- R Core Team (2025). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. https://www.R-project.org/
- Wickham, H., et al. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686. https://doi.org/10.21105/joss.01686
- Lawrence, M. A. *ez: Easy analysis and visualization of factorial experiments*. R package. https://CRAN.R-project.org/package=ez
- Müller, K. *here: A simpler way to find your files*. R package. https://CRAN.R-project.org/package=here

Checklist

We used the following checklist to make sure that our work meets all the requirements and grading criteria of the assignment.

Report content

- ☒ Authors and date in the header
- ☒ Clickable link to our public GitHub repository
- ☒ Analysis of our team's Git workflow strategy
- ☒ Analysis of the challenges encountered and lessons learned
- ☒ Complete code and answers to all the questions (1.1 to 1.7)
- ☒ Data sources (with their URLs)
- ☒ Scientific references (with their DOIs)
- ☒ Checklist used to meet the grading criteria

ANOVA pipeline and Quarto document

- ☒ Clear ANOVA pipeline, with an explanation of each step
- ☒ Master Quarto document (`IEAP-Series03-Rstudio.qmd`) in which the sub-documents are included when rendering
- ☒ Graph similar to Figure 3 of the article, with regression lines and equations
- ☒ Graph exported as an 8×6 inch PDF file

GitHub repository

- ☒ Public repository named `IEAP-Series03-RStudio`
- ☒ `main` branch up to date
- ☒ Clear and complete `README.md`
- ☒ `LICENSE` file
- ☒ Rendered PDF file of the report
- ☒ Clear Git history with meaningful commit messages
- ☒ At least one commit per question of the report
- ☒ Use of branches to work on different questions in parallel, merged into `main` with Pull Requests